SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces LebihMudah.my, an agentic AI-based solution designed to improve the traditional property rental experience. The system addresses the limitations of manual searching and static listings by enabling users to interact natural language, allowing the AI to understand requests, perform actions, and coordinate workflows dynamically. It transforms a conventional rental platform into a stateful, intelligent orchestration system capable of handling complex, real-world scenarios.

1.1. Purpose:

The system analysis document outlines the technical scope, architecture, and key design decisions behind the development of LebihMudah.my. It serves as a guide to explain how the system leverages AI and modular architecture to deliver an intelligent rental assistant.
So, the key element of system that has been covered are as followed:

1. **Architecture**:
   The system is divided into two main layers: a platform layer (Next.js and Node.js) for user interface and APIs, and an agentic core powered by Z AI GLM, responsible for reasoning, decision-making, and workflow orchestration.
2. **Data Flows:**
   User input flows from the chat interface through an ingestion router, where it is standardized and processed by the AI. The system then interacts with internal APIs and returns structured, conversational responses to the user.
3. **Model Process:**
   The model follows a React-based workflow, where it interprets input, reasons about the next step, executes tool actions, and handles multi-step interactions such as searching, clarifying, booking, and resuming workflows.
4. **Role of Reference:**
   This document acts as a reference for developers and stakeholders to understand the system's architecture, workflows, and AI behavior, ensuring consistent implementation and evaluation.

1.2. Background:

© UMHackathon 2026

Email: umhackathon@um.edu.my

LebihMudah.my is developed to address inefficiencies in the property rental process, where users often rely on manual searches and incomplete listings. The system introduces an AI-driven approach to simplify interactions and automate decision-making.

1. Previous Version:
   Traditional property rental platforms primarily function as static listing systems, where users manually filter properties and directly contact owners. These systems lack intelligent assistance, real-time interaction, and workflow continuity.

2. Changes in Major Architectural Components:
   The new system introduces an agentic AI core that separates reasoning and execution from the frontend platform. It incorporates stateful memory, tool-based APIs, and asynchronous communication, significantly enhancing system capabilities.

3. New capabilities that are only introduced in this:
   The system enables natural language interaction, automated property search, intelligent clarification, and asynchronous communication with property owners. Most importantly, it introduces stateful workflows, allowing conversations to pause and resume seamlessly, which is not supported in traditional platforms.

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Customer (Renter / Property Seeker)	Browse properties, apply filters (location, budget, rooms, pet policy), request bookings, and track booking status through the platform.	Simple and intuitive property search experience Accurate and fast filtering of listings Secure booking and authentication process Real-time booking status updates (Pending / Confirmed / Cancelled) Clear property details with availability information
Property Owner	Manage listed properties, receive booking requests, and approve or reject	Immediate notification of booking requests

© UMHackathon 2026

	bookings through the owner dashboard.	Clear booking details (renter info, dates, property ID) Easy-to-use dashboard for managing properties Ability to update property availability and details Secure access control (owner-only features)
Development Team	Design, build, and maintain the LebihMudah.my platform (frontend, backend, and database integration).	• Clear API contracts for all services (auth, search, booking, owner tools) • Modular and maintainable codebase (Next.js 14 + Prisma architecture) • Well-documented system architecture • Smooth integration between frontend, backend, and database layers • CI/CD pipeline with automated testing and validation

© UMHackathon 2026

Email: umhackathon@um.edu.my

System Architecture & Design

High Level Architecture

Overview

Type	Details
System	Web-based Property Search & Booking Platform (Next.js 14)
Architecture	Monolithic Web Application with API Layer + AI Service Integration (RAG-style chatbot)

System Description

LebihMudah.my is a client-server-based property platform consisting of:

1. Client Layer

- **Web Application (Next.js 14)**
  - Property search interface
  - Booking interface
  - Owner dashboard (/dashboard)
  - Authentication pages (/login, /signup)
  - Chatbot interface (/chat)

2. Backend API Layer (Next.js API Routes)

All business logic is handled through API routes:

- /api/auth/* → Authentication system
- /api/tools/search → Property search engine
- /api/tools/book → Booking system
- /api/owner/* → Owner management system
- /api/renter/* → Renter booking history system

3. Database Layer

- Prisma ORM
- SQLite (local development database)
- Stores:

© UMHackathon 2026

- Users (renters & owners)
- Properties
- Bookings
- Booking statuses

4. AI Service Layer (GLM Integration – Chatbot System)

The system integrates GLM as a context-aware assistant layer, not a standalone AI module.

- Used in /chat route
- Acts as a query interpreter + tool selector

LLM as Service Layer (GLM Integration Design)

How GLM is Used

GLM is integrated as a middleware reasoning layer between user input and backend tools.

Prompt Construction Flow

Step 1: User Input

Example:

"Find me a condo under RM2000 near KLCC"

Step 2: Context Injection

System builds prompt with:

- User message
- User role (guest / renter / owner)
- Session history (if logged in)
- Available tools:
  - search API
  - booking API

Step 3: Prompt Format Sent to GLM

Example structure:

SYSTEM: You are a property assistant for LebihMudah.my.

© UMHackathon 2026

TOOLS AVAILABLE:
- search_properties(location, budget, rooms)
- create_booking(property_id)

USER INPUT:
Find me a condo under RM2000 near KLCC

Context Window Contents

The GLM receives:

- User query
- Session state (if logged in)
- Tool definitions
- System instructions
- Optional previous chat history

Response Handling

GLM output is parsed into:

1. Intent Detection

- Search request
- Booking request
- Invalid/unsafe request

2. Structured Action Output

Example:

```
{
 "action": "search_properties",
 "location": "KLCC",
 "budget": 2000,
 "rooms": null
}
```

3. Backend Execution

- API routes execute actual database queries
- Prisma fetches real property data
- Results returned to frontend

© UMHackathon 2026

Email: umhackathon@um.edu.my

Token Limit & Input Control

- Max input size enforced at API layer (~1500 tokens)
- Large inputs:
  - truncated OR chunked before sending to GLM
- Prevents:
  - overflow errors
  - unnecessary cost spikes

Major API Calls Flow

- Frontend → /api/tools/search
- Frontend → /api/tools/book
- Chatbot → /api/chat (GLM gateway)
- Backend → Prisma → SQLite DB
- Owner Dashboard → /api/owner/*

Sequence Diagram (User Journey: Property Booking Flow)

Flow: User Books a Property

1. User searches property
2. System returns results
3. User selects property
4. Booking request created
5. Owner approves booking
6. System updates status

Sequence Steps

1. User → Frontend (Search Page)
2. Frontend → /api/tools/search
3. Backend → Database (Prisma)
4. Database → Backend returns results
5. User selects property
6. Frontend → /api/tools/book
7. Backend → Booking stored (status = Pending)
8. Owner logs in → /dashboard
9. Owner → /api/owner/bookings
10. Owner approves booking

© UMHackathon 2026

Email: umhackathon@um.edu.my

11. Backend updates DB (status = Confirmed)
12. User sees updated status

Technology Stack

Frontend

- Next.js 14 (App Router)
- TypeScript
- Tailwind CSS

Backend

- Next.js API Routes
- Prisma ORM

Database

- SQLite (development)
- Prisma-managed schema

AI Layer

- GLM (Chatbot reasoning engine)
- Tool-based structured output (RAG-style logic)

Deployment

- Local development: Node.js server
- CI/CD: GitHub Actions (planned/partial)
- Future deployment: Vercel / Cloud hosting

Key Data Flows

Data Flow Overview

1. User → System Flow

- User sends request (search / booking)
- API processes request

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Database returns structured data
- UI renders results

2. Booking Flow

- Request → API → Prisma → DB
- Status updates:
  - Pending → Confirmed → Cancelled

3. AI Chat Flow

- User input → GLM
- GLM → structured intent
- Intent → API execution
- API → DB results → response back

Data Flow Characteristics

- No external black holes (all requests have return paths)
- Fully traceable API-to-DB flow
- Stateless frontend, state managed in DB

Normalized Database Schema (3NF Design)

Entities

Users

- id (PK)
- name
- email
- password
- role (renter / owner)

Properties

- id (PK)
- title
- location
- price
- rooms
- owner_id (FK → Users)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Bookings

- id (PK)
- user_id (FK → Users)
- property_id (FK → Properties)
- status (Pending / Confirmed / Cancelled)
- created_at

Relationships

- One User → Many Bookings
- One Owner → Many Properties
- One Property → Many Bookings

Normalization Notes

- No duplicate property data in bookings
- No repeated user info in properties
- Booking acts as junction between Users and Properties

Integrity Rules

- Foreign key constraints enforced via Prisma
- Booking cannot exist without valid user + property
- Owner validation enforced at API layer

Summary

- System uses monolithic Next.js architecture with modular API routes
- GLM is integrated as a tool-driven reasoning layer (not free-form AI)
- Data flows are fully traceable from UI → API → DB
- Database is normalized and relationship-driven
- Booking system is the core transactional pipeline

2. Functional Requirements & Scope

.

2.1. Minimum Viable Product:

2.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description

© UMHackathon 2026

1	AI-Powered Property Search	This feature allows users to search for rental properties using natural language through a chatbot interface. Instead of using traditional filters, users can type queries such and the system will automatically interpret the request. The Agentic AI Core extracts key information like location, budget, and room requirements, then calls the appropriate backend API to retrieve matching properties.
2	Dynamic Clarification & Multi-Step Reasoning	The system is designed to handle incomplete or ambiguous user input by engaging in multi-step reasoning. When essential information is missing, such as budget or move-in date, the AI will ask follow-up questions to refine the search criteria. It maintains the context of the conversation and progressively improves the results based on user responses.
3	. Stateful Owner Inquiry	When certain property details are unavailable in the database, the system initiates a query to the property owner instead of making assumptions. It sends a message and temporarily pauses the user's workflow while saving the session state. Once the owner responds, the system resumes the conversation seamlessly and delivers the updated information to the user.
4	Property Booking Integration	The platform enables users to complete the booking process directly within the chatbot interface. After selecting a suitable property, users provide necessary details such as contact information and preferred move-in date. This integration ensures a smooth end-to-end experience.

2.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	The system must handle multiple concurrent users performing property searches and bookings, especially during peak usage periods.	Deploy services using scalable cloud infrastructure, cloud VM and React .Use horizontal scaling

© UMHackathon 2026

		and API-based architecture to distribute load efficiently.
Reliability	The system should ensure high availability and prevent data inconsistency, especially during booking transactions and session handling.	Use transactional database operations for booking processes and implement error handling with retries. Session states are stored in a persistent memory store,SQLite to prevent data loss.
Usability	The system must provide an intuitive and user-friendly experience for users with minimal technical knowledge.	Design a simple chatbot interface with clear prompts, guided responses, and error messages to assist users throughout the interaction.
Token Latency	The AI response time should be fast enough to maintain a smooth user experience, ideally within 3–5 seconds under normal load.	Implement asynchronous API calls to the GLM service with timeout control. Display loading indicators in the UI while waiting for responses and optimize prompt size to reduce processing time.
Cost Efficiency	The system should minimize API token usage and infrastructure costs per user session.	Optimize prompt engineering by limiting context size, summarizing past conversations, and caching repeated queries. Use lightweight databases during MVP stage to reduce infrastructure cost.

2.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a. Advanced Recommendation System
An AI-Driven recommendation engine that suggests properties based on user preferences, past searches, and behavioral patterns. This would enhance personalization beyond basic search functionality.

b. Notification and Alert System
A real-time notification system to inform users about booking confirmations, owner replies, new property listings, and reminders. Notifications can be delivered via email, push notifications, or in-app alerts.

© UMHackathon 2026

c. Payment Gateway Integration
Integration with secure online payment systems such as FPX ,debit cards or e-wallets to enable users to complete rental payments directly within the platform. This includes transaction handling, payment verification, and receipt generation.

3. Monitor, Evaluation, Assumptions & Dependencies

3.1. Technical Evaluation:

3.1.1. Grayscale Rollout & A/B Testing

The team will adopt a phased deployment strategy with a primary focus on the Agentic Core to ensure system stability and performance. Initially, a grayscale rollout approach will be applied where the “Stateful Pause” feature allowing the AI to pause workflows while waiting for property owner responses will be released to only 5% of users. During this phase, the system will be closely monitored to ensure that the State Memory Store can correctly assign and restore session id contexts without any data loss or inconsistencies. Once the feature is stable, A/B testing will be conducted on the React Engine’s system prompts by comparing two variants. Variant A, which delivers direct technical responses, and Variant B, which provides more friendly and conversational summaries.

3.1.2. Emergency Rollback (ER) & Golden Release

To ensure platform resilience, an Emergency Rollback (ER) and Golden Release strategy is implemented. This mechanism is triggered when the Tool Execution Sandbox detects a high rate of GLM hallucinations, particularly when invalid Parameter JSON outputs bypass validation and threaten system reliability. Once triggered, the automated deployment pipeline will immediately revert the Agentic Core to a stable Golden Version, which operates as a stateless, traditional search-and-respond system without relying on the complex React reasoning loop. During this rollback process, traffic is redirected away from the Python/Node.js Agentic Core and routed directly to the stable Next.js Platform Layer APIs. This ensures that core functionalities, such as property search, remain available to users while minimizing system disruption.

3.1.3. Priority Matrix:

The Priority matrix for the Lebih Mudah.my project is structured around the stability of the Agentic Core and the reliability of the State Memory Store. P1 Critical issues are defined by systemic failures in the initiate_booking tool or the corruption of the session persistence layer. If the failure rate for booking exceeds 1% or if session_id data is lost ,preventing the system from resuming paused workflows and the system will trigger an Emergency Rollback to the "Golden Release" to maintain the platform's basic search functionality.

P2 High priority triggers involve failures in the React Engine's orchestration or the message_owner tool's asynchronous "wake up" logic. If the Tool Execution Sandbox

© UMHackathon 2026

detects persistent parameter hallucinations or if the webhook fails to restore conversation context after an owner's reply, the system will automatically disable the "Stateful Pause" flow. In this scenario, the AI is instructed to handle the failure gracefully by informing the user that the owner is unreachable and suggesting alternative properties with verified amenities.

P3 Medium priorities focus on data-level inaccuracies or empty returns from the search_properties tool. If a search for a specific location, such as "near UM," returns no results, the system will not crash; instead, it follows the MVP Scope rules to apologize and prompt the user to refine their search parameters, such as adjusting the budget or room count. This approach ensures that the "Brain" service remains resilient even when facing unstructured input or database gaps.

3.2. Monitoring:

1. Monitoring Objectives

The monitoring system is designed to:

- Ensure system availability (>99% uptime target in production)
- Detect performance degradation early (latency spikes, API slowdowns)
- Identify critical system failures (booking/auth failures)
- Monitor data consistency in booking lifecycle
- Track AI chatbot errors or abnormal responses

2. Monitoring Architecture

Overview

The system uses a layered monitoring approach:

1. Application Layer Monitoring

- API response tracking (/api/*)
- Error rate tracking (4xx / 5xx responses)
- Authentication failures

2. Database Monitoring

- Prisma query performance logs
- SQLite integrity checks (dev)
- Transaction failure detection (booking system)

3. Frontend Monitoring

© UMHackathon 2026

- Page load time (Next.js routes)
- Failed API calls from UI
- User navigation errors

4. AI Layer Monitoring (GLM Chatbot)

- Invalid or unstructured responses
- Tool-call failure rate
- Hallucination detection flags

3. Key Monitoring Metrics

Metric	Threshold	Action Trigger
API Response Time	> 800ms	Warning Alert
API Failure Rate	> 5%	Critical Alert
Booking Failure Rate	> 2%	Immediate Alert
Authentication Failure Spike	> 10/min	Security Alert
Database Query Delay	> 500ms	Warning Alert
AI Invalid Response Rate	> 15%	Model fallback triggered
System Uptime	< 99%	Critical Incident

4. Monitoring Tools & Strategy

Local Development

- Console logs (Next.js server logs)
- Prisma query logging
- Browser DevTools (Network + Performance tabs)

© UMHackathon 2026

Email: umhackathon@um.edu.my

CI/CD Monitoring

- GitHub Actions logs:
  - Build failures
  - Test failures
  - API regression failures

Production Monitoring (Planned / Scalable Design)

If deployed:

- Vercel Analytics / Server logs
- Sentry (error tracking)
- Uptime monitoring (Ping-based checks)
- Database monitoring via Prisma logging layer

5. Alerting System Design

Alert Levels

🟢 INFO

- Normal system activity
- Logged only (no action required)

🟡 WARNING

Triggered when:

- API response slows (>800ms)
- Minor error rate increase
- Chatbot inconsistent response

Action:

- Log issue
- Notify development dashboard

🔴 CRITICAL

Triggered when:

- Booking system fails

© UMHackathon 2026

- Authentication breakdown
- Database errors
- AI returns invalid structured output repeatedly

Action:

- Immediate system alert
- Block affected feature if needed
- Rollback deployment (CI/CD gate integration)

6. Health Check System

A dedicated health endpoint is used:

GET /api/health

Returns:

- API status
- Database connectivity
- Authentication service status
- Booking service status

Example response:

```
{
 "status": "healthy",
 "api": "ok",
 "database": "connected",
 "auth": "stable",
 "booking": "operational"
}
```

7. Failure Detection Strategy

1. API Failure Detection

- Monitors HTTP status codes

© UMHackathon 2026

- Flags repeated 500 errors
- Automatically logs failed endpoints

2. Booking System Monitoring

- Detects:
  - Missing status updates
  - Duplicate bookings
  - Failed transactions

3. Authentication Monitoring

- Tracks:
  - Login failure spikes
  - Invalid session usage
  - Unauthorized access attempts

4. AI Chat Monitoring (GLM)

- Validates:
  - Structured output format
  - Tool-call correctness
  - Hallucination probability

*Fallbac*k:

- If failure detected → system switches to:
  - rule-based response mode OR
  - search-only mode (no generation)

8. Incident Response Flow

When a critical issue is detected:

1. Alert triggered (CI/CD or runtime)
2. Logs captured (API / DB / AI layer)
3. Issue classified:
   - API issue
   - Database issue
   - AI failure
4. System mitigation:
   - rollback (if deployment-related)
   - feature disable (graceful degradation)

© UMHackathon 2026

5. Developer notified via CI/CD logs

9. Summary

- Monitoring is multi-layered (frontend, backend, DB, AI)
- System uses threshold-based alerting
- Critical failures trigger automatic blocking or rollback
- AI system is monitored for structured output correctness
- Booking system is treated as a high-priority critical module

3.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment.

a. Users have stable internet connections on their devices during
b. Renters have stable internet access to allow the Omni-Channel Ingestion Router to standardize their unstructured queries into a uniform JSON format for the GLM.
c. Property owners are available to use the owner Simulator or Hidden UI panel to provide real-time manual inputs.
d. Users can successfully navigate the authentication gates, such as verifying their role via the /api/auth/me endpoint before the chatbot grants access to renter or owner tools.

3.4. External Dependencies: Do list all the external tools that are applied to your system to function.

Tools	*Purpose*	*Risks*
Z Ai GLM API	This serves as the core "Brain" of the system. It handles unstructured renter queries, manages multi-step reasoning through the React loop, and generates the ParameterJSON required to trigger tools like search_properties or initiate_booking.	High. Beyond rate limits, the system faces risks of "hallucinations" where the GLM might guess property details.

© UMHackathon 2026

Next.js & Node.js Infrastructure	This is the Body that houses the internal API routes and the mock database. It provides the UI shell and manages user sessions and role-based access.	Medium. Issues with session persistence or role verification via the /api/auth/me endpoint could block authenticated actions like booking or property editing.
Stateful Webhook Service	Facilitates the "Stateful Pause" and "Wake Up" logic. It allows the Agentic Core to pause a rental workflow and resume it once the Owner Simulator sends a response.	High. A failure in the webhook delivery prevents the AI from resuming a conversation after a third-party consultation.
Google Maps Platform	Used for property discovery and location visualization. It allows the chatbot to process location-based queries provides the geographical context needed	Medium. If the API service is interrupted, the agent may struggle to parse specific neighborhood distances or display property locations on the UI.

Project Management & Team Contributions

The LebihMudah.my Property Platform was developed following an Agile 10-day sprint model aligned with the preliminary hackathon round timeline. The project was divided into structured phases to ensure continuous delivery, testing, and integration of core modules (search, booking, authentication, owner dashboard, and AI chatbot).

---

1. Project Timeline (10-Day Sprint Plan)

Day 1–2: Project Planning & Requirement Analysis

- **Define system scope (renter, owner, chatbot)**
- **Identify core features (search, booking, dashboard, auth)**
- **Design initial architecture (Next.js + Prisma + GLM integration)**
- **Setup GitHub repository and project board**

© UMHackathon 2026

Email: umhackathon@um.edu.my

Justification:
Ensures clear scope definition and avoids feature creep early in development.

Day 3–4: System Design & Database Setup

- **Design Prisma schema (Users, Properties, Bookings)**
- **Setup SQLite database**
- **Define API structure (/auth, /tools, /owner, /renter)**
- **Seed initial dataset**

Justification:
Strong database foundation ensures consistency in booking and property workflows.

Day 5–6: Backend & API Development

- **Implement authentication system (login/signup/session)**
- **Build property search API**
- **Develop booking system logic**
- **Implement owner dashboard APIs**

Justification:
Core business logic is prioritized before UI and AI integration.

Day 7: Frontend Integration

- **Build property listing UI**
- **Implement search filters**
- **Connect frontend with backend APIs**
- **Develop booking UI flow**

Justification:
Ensures full vertical slice (frontend ↔ backend integration).

Day 8: AI Chatbot (GLM Integration)

- **Integrate GLM chatbot in /chat**
- **Implement prompt construction layer**
- **Add tool-based response system (search + booking intent detection)**

Justification:
AI layer enhances user experience and simulates agentic system behavior.

Day 9: Testing & CI/CD Setup

- **Unit testing (auth, booking, search)**

© UMHackathon 2026

- **Integration testing (full booking flow)**
- **Setup GitHub Actions pipeline**
- **Run regression tests**

Justification:
Ensures system stability and prevents deployment of broken features.

Day 10: Final Optimization & Documentation

- **Performance tuning (API response optimization)**
- **Bug fixing**
- **Write test strategy + architecture documentation**
- **Final submission preparation**

Justification:
Ensures production readiness and complete hackathon documentation.

2. Team Members & Roles

Backend & System Lead

- API design (Next.js API routes)
- Prisma schema design
- Booking system logic
- Authentication system
- Database integrity management

Frontend Developer

- Next.js UI development
- Property listing pages
- Booking interface
- Owner dashboard UI
- Responsive design (Tailwind CSS)

AI / Integration Engineer

- GLM chatbot integration
- Prompt engineering & tool routing logic
- Context window management
- Hallucination control strategy

© UMHackathon 2026

Email: umhackathon@um.edu.my

QA / Testing Engineer

- Unit and integration testing
- Edge case validation (booking, auth, search)
- Performance testing (API load simulation)
- Regression testing across features

DevOps / CI-CD Engineer

- GitHub Actions pipeline setup
- Build automation and test execution
- Deployment readiness checks
- Monitoring strategy setup

3. Project Management Approach

The project follows an Agile Sprint-Based Model:

Key Principles:

- Daily incremental progress
- Continuous integration via GitHub
- Feature-first development (vertical slices)
- Testing embedded throughout development (not at end)

Workflow:

1. Feature planning (backlog)
2. Implementation (branch-based development)
3. Testing (unit + integration)
4. Merge into main branch via CI/CD gate

4. Recommendations (Scalability, Performance & Reliability)

1. Caching Layer (Redis)

- Cache frequently accessed properties
- Reduce repeated database queries for search results
- Improve response time for /api/tools/search

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Database Optimization

- Move from SQLite → PostgreSQL for production scaling
- Add indexing on:
  - location
  - price
  - property_id
- Optimize booking query performance

3. AI Optimization (GLM Layer)

- Introduce response caching for repeated queries
- Use tool-based strict prompting (reduce hallucination)
- Implement fallback mode (search-only response mode)

4. Load Balancing (Future Scaling)

- Deploy backend on multiple instances (Vercel / AWS)
- Use API Gateway for request distribution

5. Monitoring & Observability

- Integrate real-time logging (Sentry / Vercel Analytics)
- Add performance dashboards (API latency, booking success rate)
- Alert system for critical failures

6. Security Enhancements

- Strengthen RBAC (Role-Based Access Control)
- Encrypt sensitive session data
- Add rate limiting for APIs (prevent abuse)

5. Summary

- Development followed a 10-day Agile sprint structure
- Clear separation of roles ensured parallel development
- System built using modular full-stack architecture
- AI integration enhances user interaction but remains tool-controlled
- Scalability improvements identified for production readiness

© UMHackathon 2026

Email: umhackathon@um.edu.my